

KREACIJSKI DIZAJN PATERNI

Uvod

Kreacijski dizajn paterni koriste se za organizovanje kreiranja objekata u sistemu. Njihova svrha je smanjenje zavisnosti između klasa i izdvajanje procesa kreiranja objekata iz glavne logike sistema.

U sistemu CampusEats analizirani su različiti kreacijski paterni, a na novi klasni dijagram dodana su dva paterna koja se najbolje uklapaju u postojeći sistem: Factory Method i Builder.

1. Singleton

Singleton patern koristi se kada u sistemu treba postojati samo jedna instanca određene klase.

Najčešće se koristi za:

- konfiguraciju sistema
- logovanje aktivnosti
- upravljanje zajedničkim resursima

U sistemu CampusEats Singleton bi se mogao koristiti za klasu koja vodi evidenciju sistemskih aktivnosti, kao što su prijava korisnika, rezervacija obroka ili generisanje QR kodova.

Korišćenje

Singleton bi omogućio da cijeli sistem koristi jednu zajedničku instancu za zapisivanje važnih događaja i grešaka.

Ovaj patern nije dodat na klasni dijagram jer nije direktno povezan sa glavnim funkcionalnostima rezervacije i upravljanja obrocima koje su obrađene u ovom zadatku.

2. Factory Method

Factory Method patern koristi se kada kreiranje objekata zavisi od određenog tipa ili uslova.

Umjesto direktnog kreiranja objekata u controllerima ili drugim klasama, kreiranje se prebacuje u posebnu factory klasu.

U sistemu CampusEats ovaj patern je odabran i dodat na klasni dijagram kod kreiranja korisnika sistema. Sistem podržava više tipova korisnika:

- student
- administrator
- radnik menze
- dostavljač

Zbog toga je logično izdvojiti kreiranje korisnika u posebnu klasu.

Na dijagram su dodani:

- IKorisnikFactory
- KorisnikFactory

KorisnikFactory kreira odgovarajući objekat korisnika na osnovu odabrane uloge korisnika.

Korišćenje

Factory Method je korišten zato što pojednostavljuje kreiranje različitih tipova korisnika.

Controller ili servis ne moraju direktno znati kako se kreira određeni korisnik, nego samo pozivaju factory klasu.

3. Abstract Factory

Abstract Factory patern koristi se kada je potrebno kreirati porodice povezanih objekata bez direktnog navođenja konkretnih klasa.

U sistemu CampusEats ovaj patern bi se mogao koristiti kada bi postojalo više različitih sistema menzi, npr:

- studentska menza
- restoran
- dostava hrane
- automatizovana kuhinja

Svaki od tih sistema mogao bi imati svoje tipove menija, rezervacija i načina dostave.

Korišćenje

Abstract Factory trenutno nije dodat na klasni dijagram jer sistem nema više različitih porodica objekata koje zahtijevaju posebne fabrike.

Za trenutni sistem dovoljan je jednostavniji Factory Method.

4. Builder

Builder patern koristi se za kreiranje složenih objekata korak po korak.

Posebno je koristan kada objekat sadrži više podataka ili kada se svi podaci ne postavljaju odjednom.

U sistemu CampusEats ovaj patern je odabran i dodat na klasni dijagram kod klase Rezervacija.

Rezervacija sadrži više podataka:

- korisnika
- obrok

- datum rezervacije
- status rezervacije
- QR kod
- način preuzimanja

Zbog toga bi kreiranje rezervacije pomoću jednog velikog konstruktora bilo nepregledno.

Na dijagram je dodana klasa:

- RezervacijaBuilder

Builder postepeno postavlja podatke rezervacije i na kraju metodom build() vraća gotov objekat Rezervacija.

Korištenje

Builder je korišten zato što proces kreiranja rezervacije nije potpuno jednostavan.

Potrebno je postaviti više podataka i provjeriti ispravnost rezervacije prije njenog kreiranja.

Builder omogućava preglednije i kontrolisanije kreiranje rezervacije.

5. Prototype

Prototype patern koristi se kada se novi objekti kreiraju kopiranjem postojećih objekata.

U sistemu CampusEats ovaj patern bi se mogao koristiti za ponavljanje prethodnih rezervacija obroka.

Na primjer, korisnik bi mogao ponovo naručiti isti obrok bez ponovnog unosa svih podataka.

Korištenje

Prototype nije dodat na klasni dijagram jer trenutni sistem nije zasnovan na kopiranju postojećih rezervacija.

Za postojeći sistem Builder predstavlja bolje rješenje jer omogućava postepeno kreiranje rezervacije

Zaključak

U ovom zadatku razmotreni su osnovni kreacijski dizajn paterni i njihova moguća primjena u sistemu CampusEats.

Na novi klasni dijagram dodana su dva paterna:

- Factory Method
- Builder

Factory Method je odabran jer sistem koristi više tipova korisnika i potrebno je centralizovati njihovo kreiranje.

Builder je odabran jer je rezervacija složen objekat sa više podataka i pogodnije ju je kreirati postepeno.

Ostali kreacijski paterni, kao što su Singleton, Abstract Factory i Prototype, također imaju moguću primjenu u sistemu, ali nisu dodani na klasni dijagram jer nisu najvažniji za trenutno proširenje sistema.